NLP

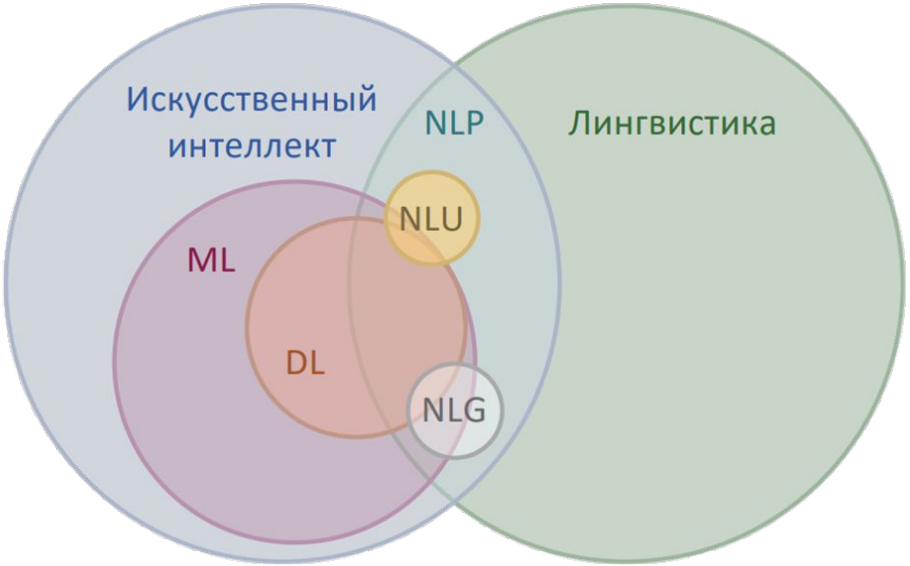
Апарин Георгий

План занятия

- Постановка задачи классификации
- Нормализация и токенизация текста
- Векторизация
- SVD разложение
- Первые нейросетевые модели

Что такое NLP

- NLP = Natural Language Processing = Обработка текстов на естественном языке
- «Естественный» — язык, используемый для общения людей
- Любые алгоритмы, связанные с обработкой и генерацией текстов
- NLU = Natural Language Understanding = Понимание естественного языка
- NLG = Natural Language Generation = Генерация естественного языка



Сложности обработки языка

- Разнообразие языков
- Свободный порядок слов
- Добавление новых слов
- Необходимость знания об окружающем мире
- Языковые особенности (анафора, неологизмы, омонимы и т. д.)

Идиомы

Мир тесен
Вышла из себя
Играть с огнем
Не май месяц

Сленг

Зашквар
Кек
Каеф
Жиза

Омонимия

Стали
Косой
Замок

Нестандартный язык

Ем можед личку
прочетаеш

NLP vs CV

Кот в России

Кот в Испании

Кот в Австралии

NLP vs CV

Кот в России

кот, кошка, котик, кошара

Кот в Испании

gato, gata

Кот в Австралии

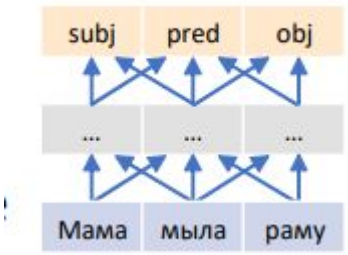
cat

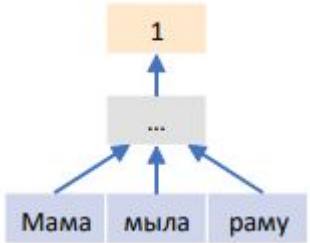
Типы задач в NLP

1. Классификация текстов

   текст → метка, несколько меток

   анализ тональности, определение темы текста





2. Классификация слов/токенов

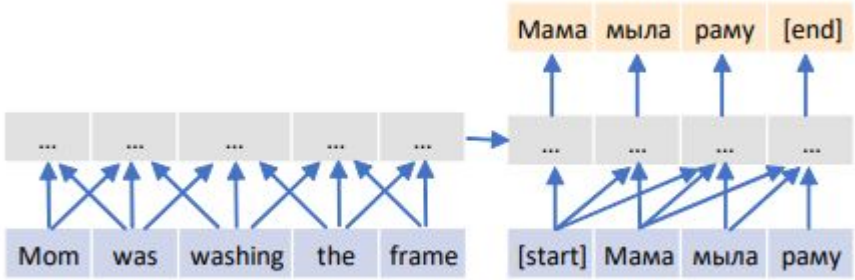
   каждое слово/токен в тексте → метка

   извлечение именованных сущностей (NER)

3. Sequence to Sequence (seq2seq)

   текст → текст

   перевод, предсказание следующего слова, фразы



Предобработка текста

- Токенизация
- Сегментация предложений
- Удаление пунктуации
- Удаление стоп-слов
- Фильтрация по длине, частоте, регулярному выражению
- Лемматизация — приведение к нормальной форме
- Стемминг — приведение к псевдооснове

Подходы к нормализации текста

Исходный текст:	Остерегайся же чересчур большого ума!
удаление пунктуации:	Остерегайся же чересчур большого ума!
удаление стоп-слов:	Остерегайся же чересчур большого ума!
лемматицация:	Остерегать же чересчур большой ума!
стемминг:	Остерегайся же чересчур большого ума!

Подходы к нормализации текста

Стемминг — определение основы слова путём отбрасывания окончаний из известного набора (возможно, псевдоосновы за счёт отбрасывания псевдоокончаний).

Лемматизация — приведение слова в начальную форму (сущ. — им. падеж, единственное число; глагол — инфинитив и т. д.)

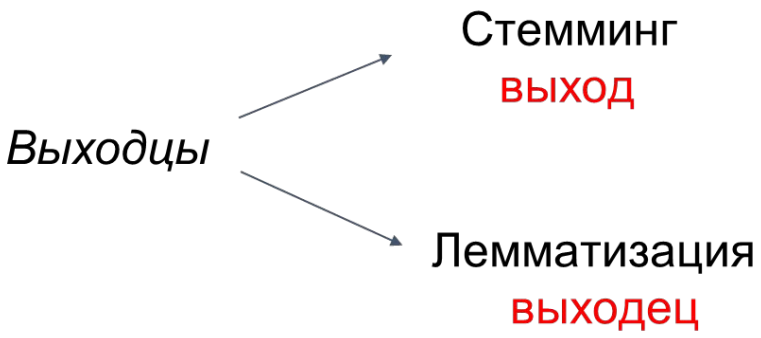
Выходцы → Стемминг ?

Выходцы → Лемматизация ?

Подходы к нормализации текста

Стемминг — определение основы слова путём отбрасывания окончаний из известного набора (возможно, псевдоосновы за счёт отбрасывания псевдоокончаний).

Лемматизация — приведение слова в начальную форму (сущ. — им. падеж, единственное число; глагол — инфинитив и т. д.)



Подходы к токенизации текста

char-level (токенизация символов)

(маленький словарь, текст разбивается на много токенов)

word-level (токенизация слов)

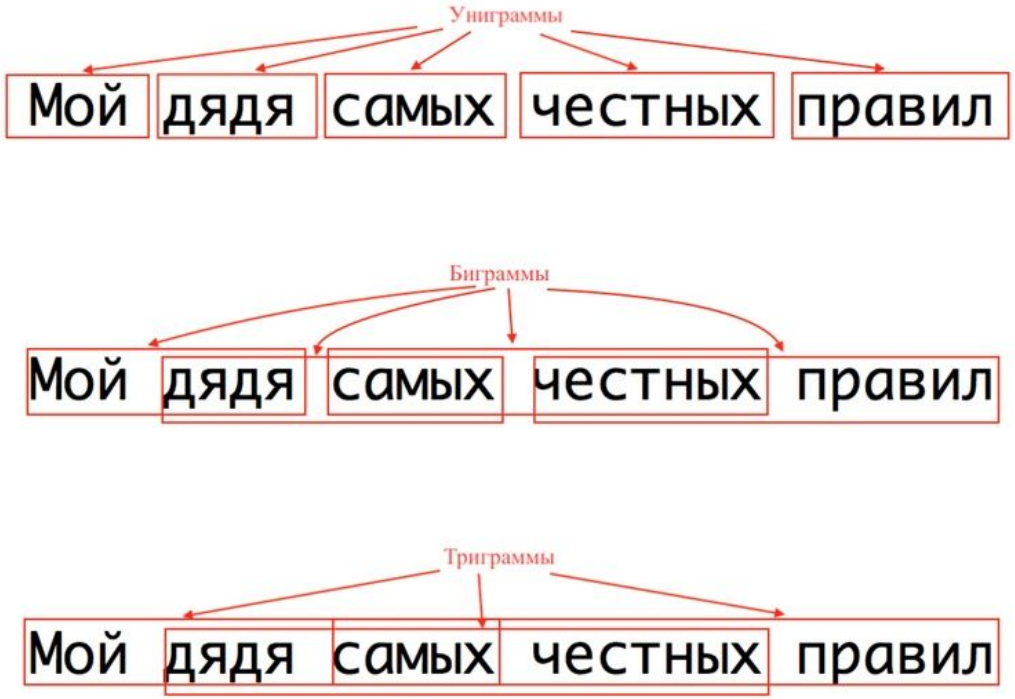
(большой словарь, текст разбивается на мало токенов)

subword-level (токенизация частей слов)

(средний словарь, текст разбивается на мало токенов)

Подходы к токенизации текста

N-Gram токенизация



Подходы к токенизации текста

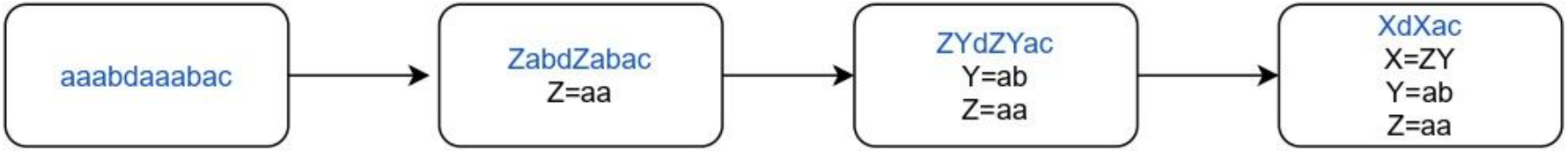
- Byte-Pair Encoding (BPE)
- WordPiece - при объединении символов в единый токен руководствуется формулой:

$$\text{score} = (\text{freq_of_pair})/(\text{freq_of_first_element} \times \text{freq_of_second_element})$$

- SentencePiece - учитывает пробелы
- Unigram - сначала строит словарь из всех униграм, затем постепенно удаляет из него менее значимые токены

Попарное битовое кодирование (BPE)

- Подсчитываем частоты встречаемости пар символов
- Заменяем самую частую пару некоторым символом, которого нет изначально в словаре
- Повторяем процедуру фиксированное количество раз



Модели векторного представления слова и текста

- Первое поколение: векторное представление текста (мешок слов, TF-IDF)
- Второе поколение: векторное представление слова (Word2vec, GloVe, fastText)
- Третье поколение: языковые модели (BERT, GPT и т. д.)

Модели векторного представления слова и текста

- Первое поколение: векторное представление текста (мешок слов, TF-IDF)
- Второе поколение: векторное представление слова (Word2vec, GloVe, fastText)
- Третье поколение: языковые модели (BERT, GPT и т. д.)

Мешок слов (Bag of Words)

- Упрощённое векторное представление текста на основе количества слов в документе.
- Задаётся словарь — все уникальные слова, которые встретились в документах коллекции.
- Представление документа — для каждого слова в словаре указывается количество вхождения этого слова в документ.
- Одно из первых упоминаний «мешка слов» в лингвистическом контексте встречается в статье 1954 года Зеллига Харриса.

Мешок слов

Текст 1
Ах, капель, капель, капель,
Золотая карусель!

Текст 2
Апрель, апрель!
На дворе звенит капель.

Мешок слов

Текст 1
Ах, капель, капель, капель,
Золотая карусель!

Текст 2
Апрель, апрель!
На дворе звенит капель.

	ах	капель	золотой	карусель	апрель	на	двор	звенеть
док 1	1	3	1	1	0	0	0	0
док 2	0	1	0	0	2	1	1	1

Представление в виде набора признаков

	ах	капель	золотой	карусель	апрель	на	двор	звенеть
док 1	1	3	1	1	0	0	0	0
док 2	0	1	0	0	2	1	1	1

$$d_1 = (f_{11}, f_{12}, \ldots, f_{1n})$$
$$d_2 = (f_{21}, f_{22}, \ldots, f_{2n})$$

Представление в виде набора признаков

	ах	капель	золотой	карусель	апрель	на	двор	звенеть
док 1	1	3	1	1	0	0	0	0
док 2	0	1	0	0	2	1	1	1

$$d_1 = (f_{11}, f_{12}, \ldots, f_{1n})$$
$$d_2 = (f_{21}, f_{22}, \ldots, f_{2n})$$

$$d_1 = (1, 3, 1, 1, 0, 0, 0, 0)$$
$$d_2 = (0, 1, 0, 0, 2, 1, 1, 1)$$

Плюсы и минусы мешка слов

Плюсы:

- Легко реализовать
- Можно применить для любого языка

Минусы:

- Не учитывается порядок слов, семантику и грамматику
- Вектор документа имеет большую размерность
- Сложность при добавлении новых слов

TF-IDF

- Статистическая мера, используемая для оценки важности слова в контексте документа, являющегося частью коллекции документов или корпуса.
- Вес некоторого слова пропорционален частоте употребления этого слова в документе и обратно пропорционален частоте употребления слова во всех документах коллекции.
- Используется в задачах анализа текстов и информационного поиска, например, как один из критериев релевантности документа поисковому запросу, при расчёте меры близости документов при кластеризации.

TF-IDF

TF (Term Frequency) — частота вхождения термина в документ, отношение числа вхождения слова в документ к общему числу слов в документе.

$$\mathrm{tf}(t, d) = \frac{n_t}{\sum_k n_k},$$

IDF (Inverse Document Frequency) — инверсия частоты, с которой некоторое слово встречается в документах коллекции.

$$\mathrm{idf}(t, D) = \log \frac{|D|}{|\{\, d_i \in D \mid t \in d_i \,\}|}$$

TF-IDF является произведением этих двух сомножителей:

$$\text{tf-idf}(t, d, D) = \mathrm{tf}(t, d) \times \mathrm{idf}(t, D)$$

TF-IDF

Текст 1
Ах, капель, капель, капель,
Золотая карусель!

Текст 2
Апрель, апрель!
На дворе звенит капель.

	ах	капель	золотой	карусель	апрель	на	двор	звенеть
док 1	0.36	0.78	0.36	0.36	0	0	0	0
док 2	0	0.26	0	0	0.73	0.36	0.36	0.36

Мешок слов и TF-IDF

- Не учитывает порядок слов
- Высокая размерность векторов
- Неограниченный диапазон значений векторов
- Не отражает семантику слов

Идея векторных представление слов. One-hot, SVD и матричная факторизация

Увеличим размерность векторов

Word	Numbers	
Apple	5	5
Soccer	0	6
House	2	2
Car	6	0

Word	Numbers			
A	-0.82	-0.32	...	-0.23
Aardvark	0.419	1.28	...	-0.06
...			...	
Zygote	-0.74	-1.02	...	1.35

4096

Увеличим размерность векторов

Как получить такие векторы слов?

Word	Numbers	
Apple	5	5
Soccer	0	6
House	2	2
Car	6	0

Word	Numbers			
A	-0.82	-0.32	...	-0.23
Aardvark	0.419	1.28	...	-0.06
...			...	
Zygote	-0.74	-1.02	...	1.35

4096

One-hot кодированиие

Преимущества:

- Простой способ получить векторное представление

Недостатки:

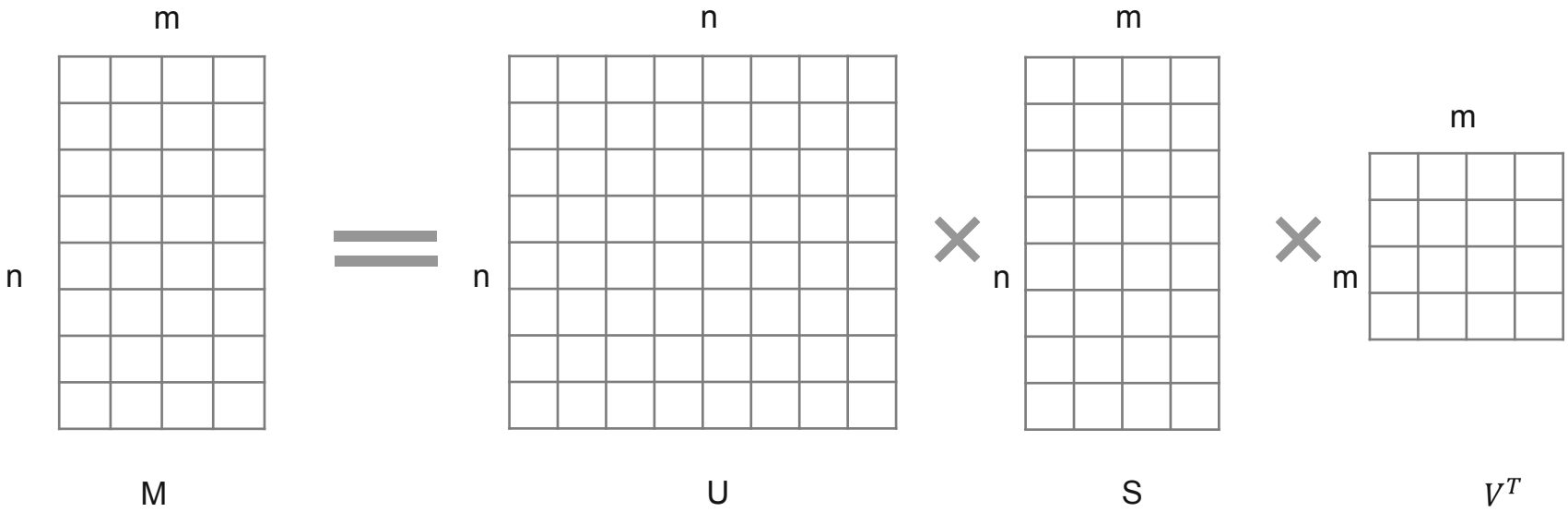
- Итоговые векторы имеют огромную нефиксированную длину
- Векторы далеки друг от друга

Слово
Apple
Soccer
House
Car

Слово	**Вектор**			
Apple	1	0	0	0
Soccer	0	1	0	0
House	0	0	1	0
Car	0	0	0	1

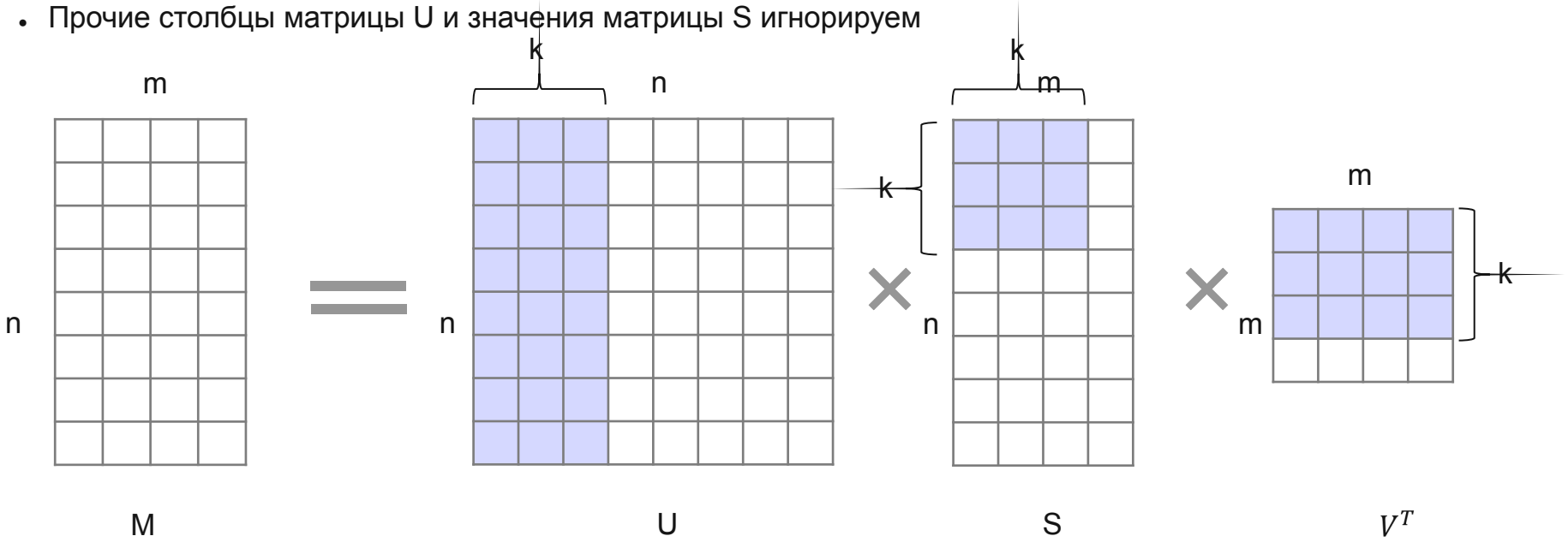
SVD

- Пусть $M \in R^{n \times m}$ - матрица мешка слов для коллекции из m документов со словарем размера n
- Применим SVD разложение: $M = USV^T$



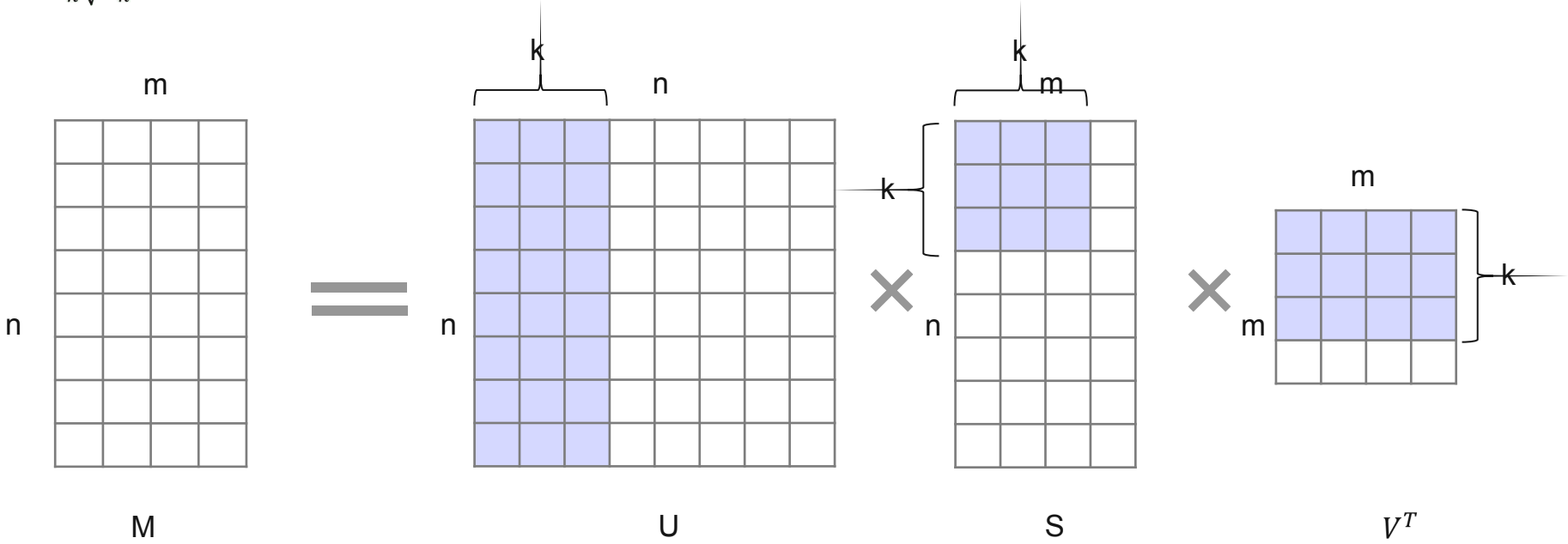
SVD

- Выберем k первых столбцов матрицы U
- k первых строк матрицы S
- Прочие столбцы матрицы U и значения матрицы S игнорируем

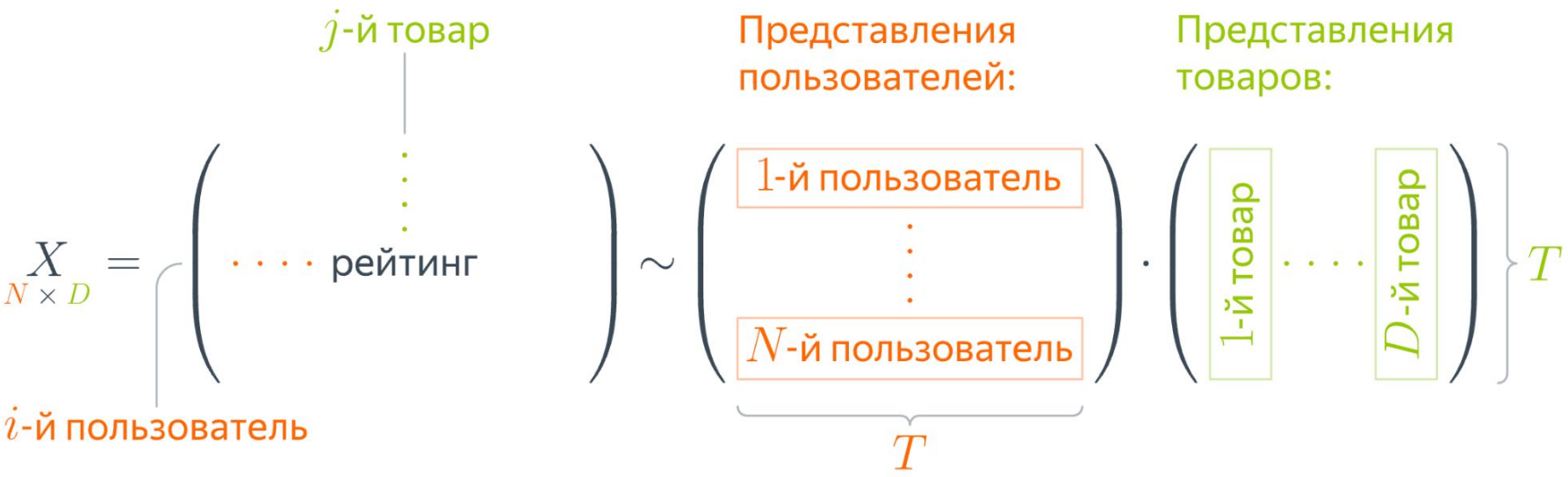


SVD

- В качестве итоговых векторных представлений слов можно взять усеченную матрицу U_k или строки матрицы $U_k\sqrt{S_k}$



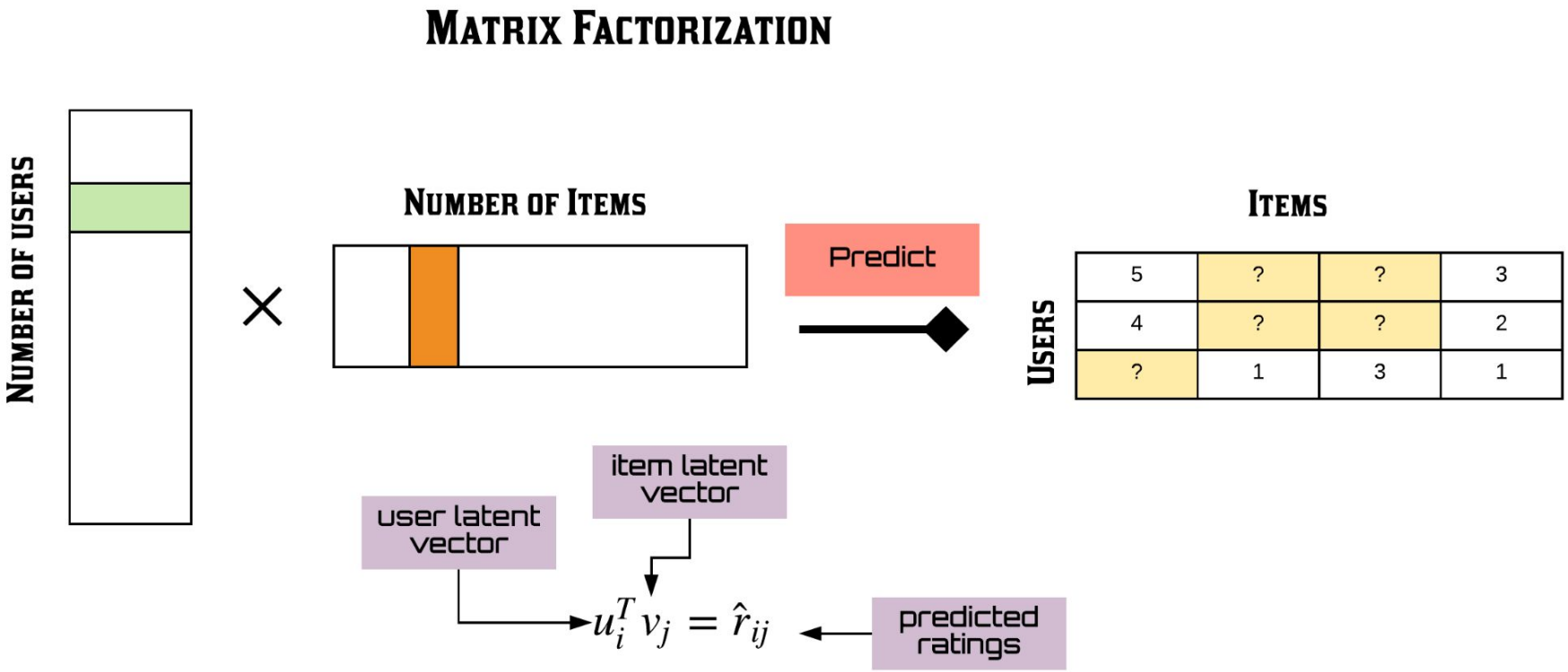
SVD


j-й товар
Представления пользователей:
Представления товаров:
$X_{N \times D} =$
рейтинг
$\sim$
1-й пользователь
N-й пользователь
1-й товар
D-й товар
T
T
i-й пользователь

SVD

$$\widehat{U}\widehat{\Sigma}^{\frac{1}{2}} \cdot \widehat{\Sigma}^{\frac{1}{2}}\widehat{V}^T$$

SVD


Matrix Factorization
Number of users
Number of Items
Predict
Items
Users
5 ? ? 3
4 ? ? 2
? 1 3 1
item latent vector
user latent vector
$u_i^T v_j = \hat{r}_{ij}$
predicted ratings

Модель Word2vec

Основная идея

Слова в схожих контекстах близки по смыслу:

- Он приоткрыл **X** и заглянул в комнату.
- Входная **X** не была заперта.
- Я открыл котику **X**, но он передумал выходить.

Какое слово скрыто под **X**?

Основная идея

Слова в схожих контекстах близки по смыслу:

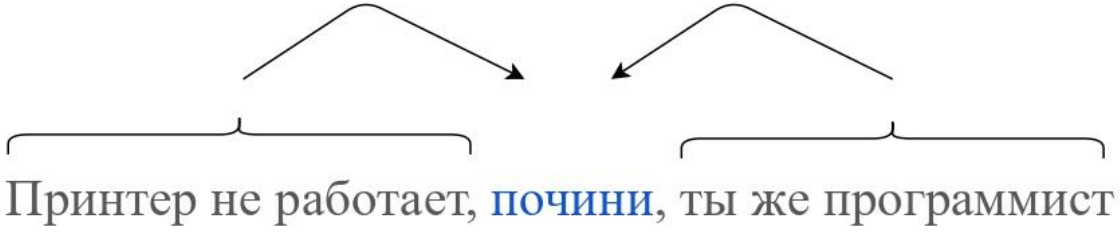
- Он приоткрыл **X** и заглянул в комнату.
- Входная **X** не была заперта.
- Я открыл котику **X**, но он передумал выходить.

Какое слово скрыто под **X**?

Дверь

CBOW и Skip-gram

Continuous Bag of Word (CBOW) — предсказывает по контексту слово:



Skip-gram — предсказывает по слову контекст:

CBOW и Skip-gram

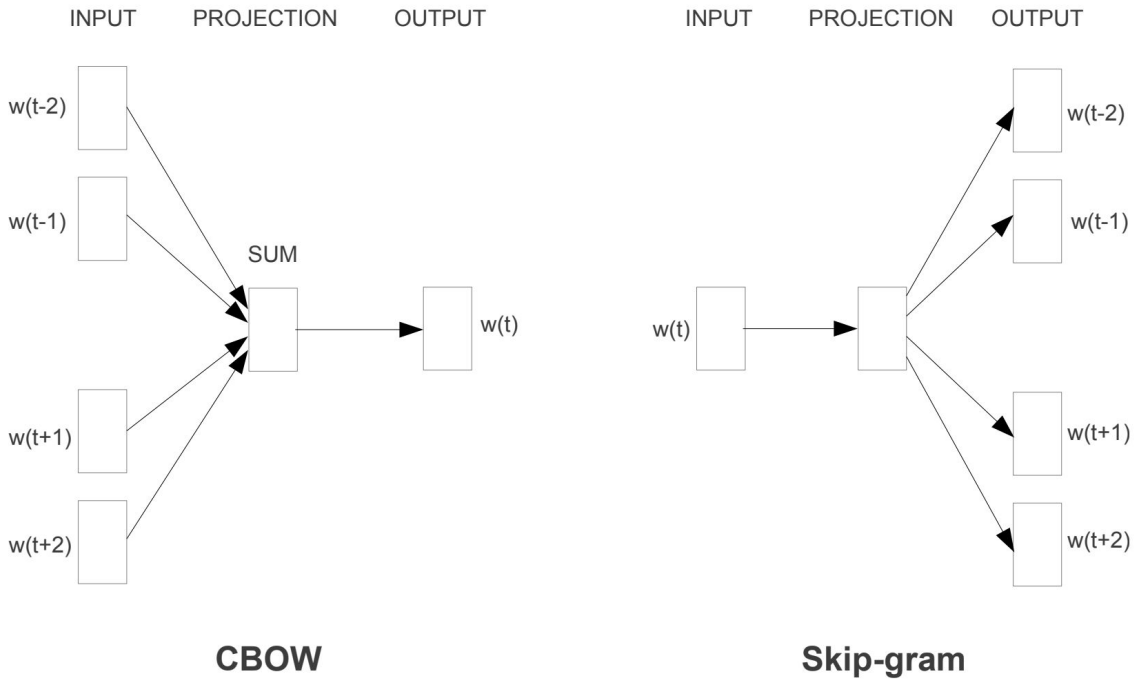


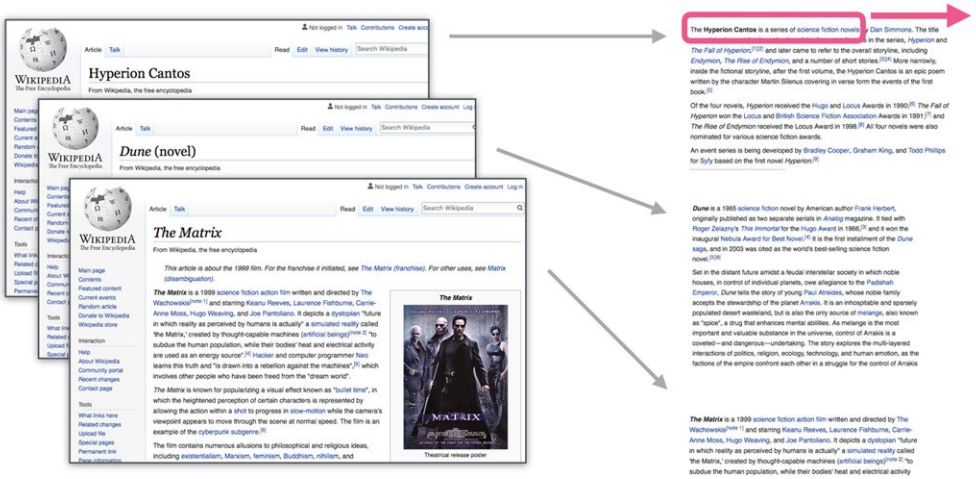
Figure 1: New model architectures. The CBOW architecture predicts the current word based on the context, and the Skip-gram predicts surrounding words given the current word.

Модель CBOW (общая схема)

- На вход сети подаются слова контекста, закодированные в one-hot представлении.
- One-hot представление — бинарный вектор длины словаря, где обозначает текущее слово.
- За одну итерацию подаются подряд идущие слова из текста заданного количества (окна).
- Задача: предсказать слово в середине.
- На следующем шаге окно сдвигается и подаются следующие n слов.
- Таким образом совершается проход по всему тексту.
- Для каждого слова запоминаются веса скрытого слоя W — это и есть векторы слов.

Обучение векторных представлений слов (данные)

- Получаем много текстовых данных (скажем, все статьи «Википедии»).
- Устанавливаем окно (например, из трёх слов), которое скользит по всему тексту.
- Скользящее окно генерирует образцы для обучения нашей модели.



Обучение векторных представлений слов (данные)

Когда мы начинаем, окно находится на первых трёх словах:

Thou shalt not make a machine in the likeness of human mind.

thou	shalt	not	make	a	machine	in	the	…

input 1	input 2	output
thou	not	shalt

Обучение векторных представлений слов (данные)

Thou shalt not make a machine in the likeness of human mind.

thou	shalt	not	make	a	machine	in	the	…

input 1	input 2	output
thou	not	shalt
shalt	make	not

Обучение векторных представлений слов (данные)

Thou shalt not make a machine in the likeness of human mind.

thou	shalt	not	make	a	machine	in	the	…

input 1	input 2	output
thou	not	shalt
shalt	make	not
not	a	make

Обучение векторных представлений слов (данные)

Thou shalt not make a machine in the likeness of human mind.

thou	shalt	not	make	a	machine	in	the	…

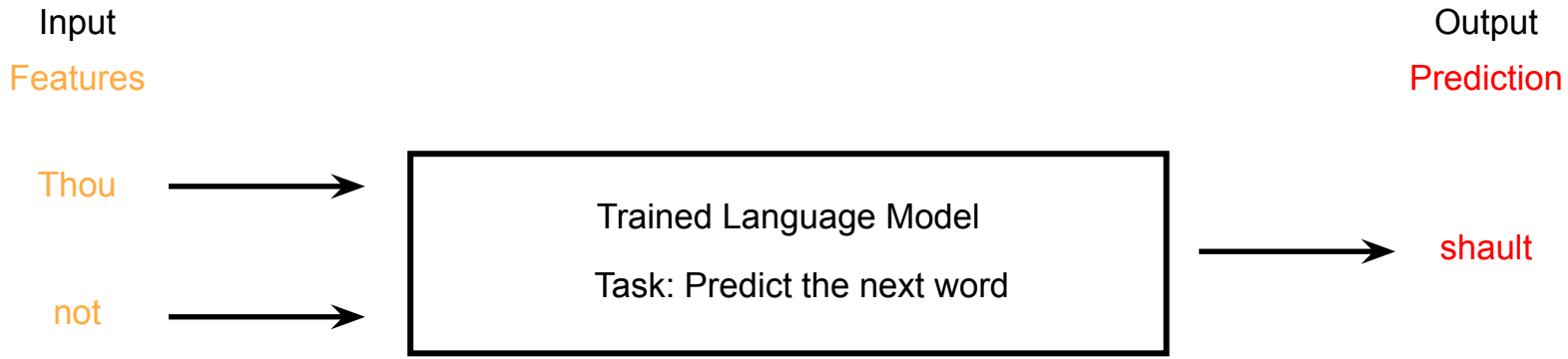
input 1	input 2	output
thou	not	shalt
shalt	make	not
not	a	make
make	machine	a

Обучение векторных представлений слов (модель)

Исходная задача:

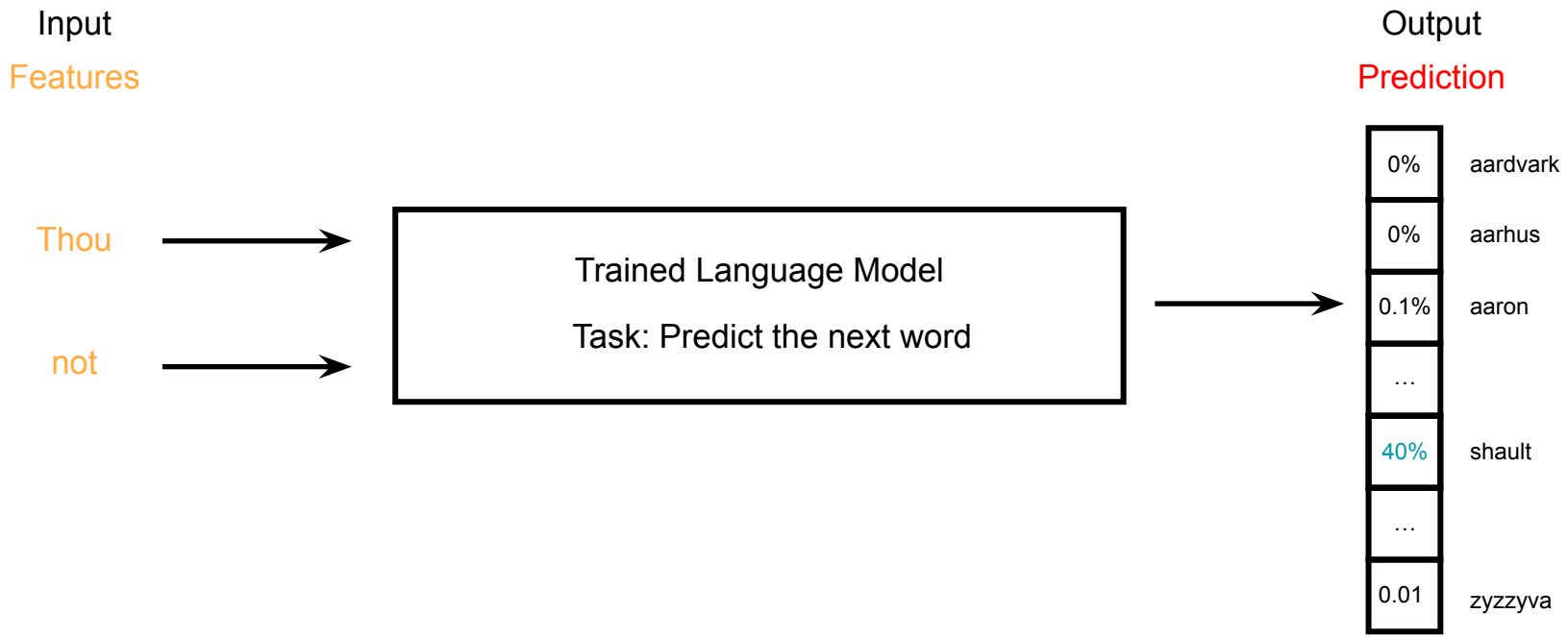
input/feature #1	input/feature #2	output/label
Thou	not	_____

Представим модель в виде чёрного ящика:

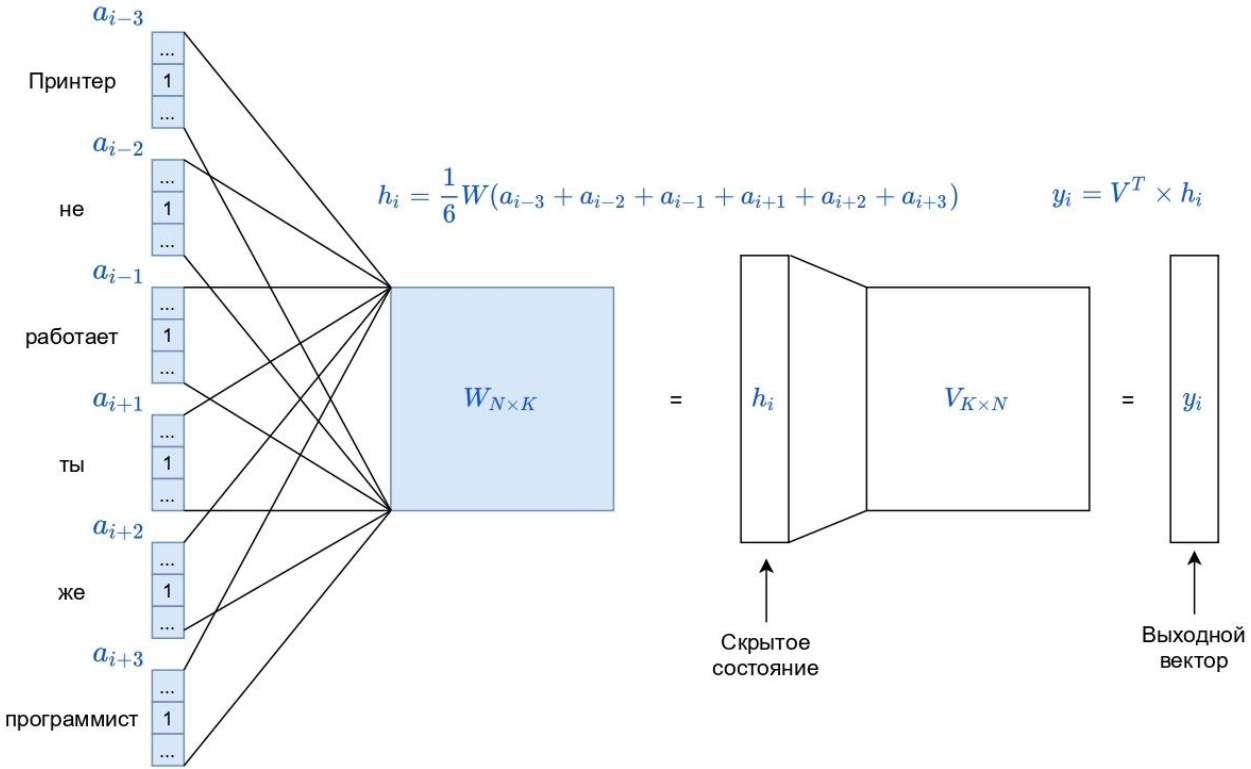


Обучение векторных представлений слов (модель)

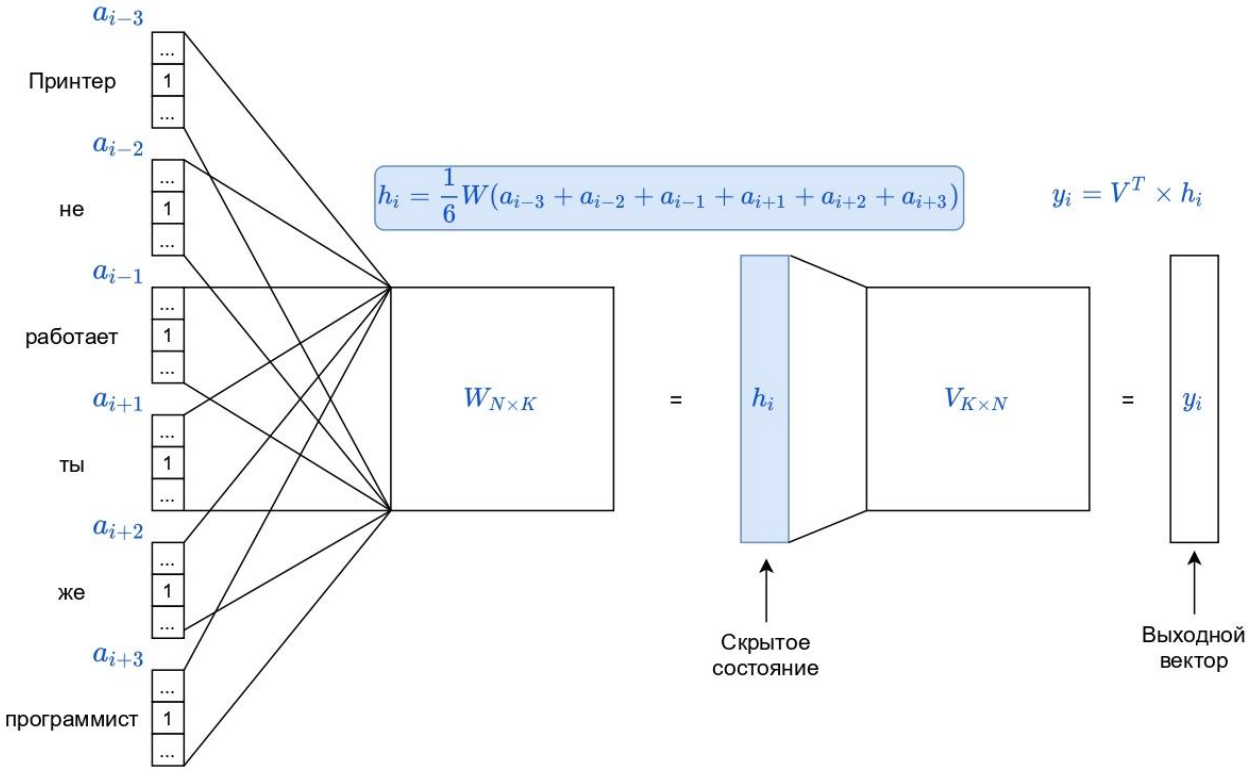
Модель выдаёт вероятностное распределение слов в словаре:



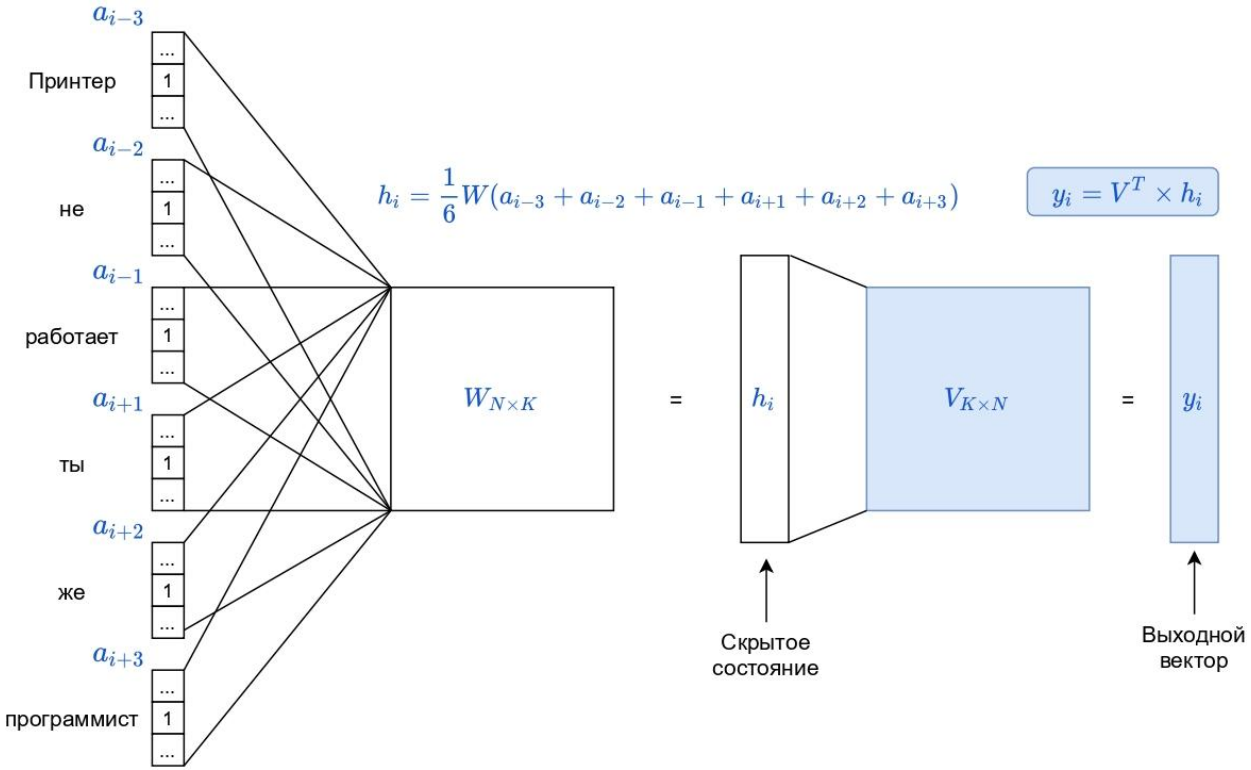
Модель CBOW (шаг 1)



Модель CBOW (шаг 2)



Модель CBOW (шаг 3)



Модель CBOW (шаг 4)

Преобразовать выходы последнего слоя в распределение вероятностей с помощью функции softmax:

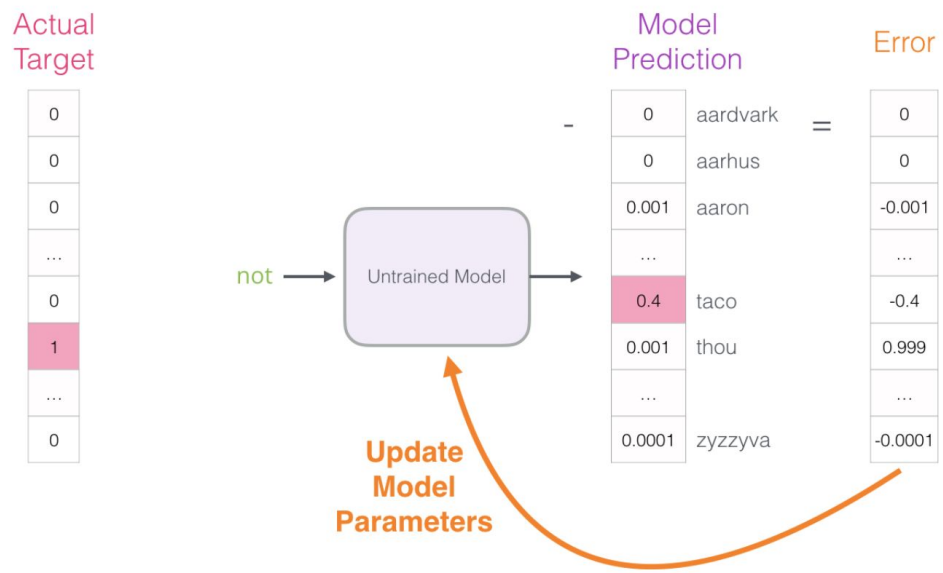
$$softmax(y) = (z_1, \ldots, z_N)$$

$$z_j = \frac{e^{y_j}}{\sum_{k=1}^{n} e^{y_k}}$$

Подсчёт ошибки

Actual Target		Model Prediction			Error
0		0	aardvark		0
0		0	aarhus		0
0		0.001	aaron		-0.001
…	-	…		=	…
0		0.4	taco		-0.4
1		0.001	thou		0.999
…		…			…
0		0.0001	zyzzyva		-0.0001

Обновление параметров



Обучение модели CBOW

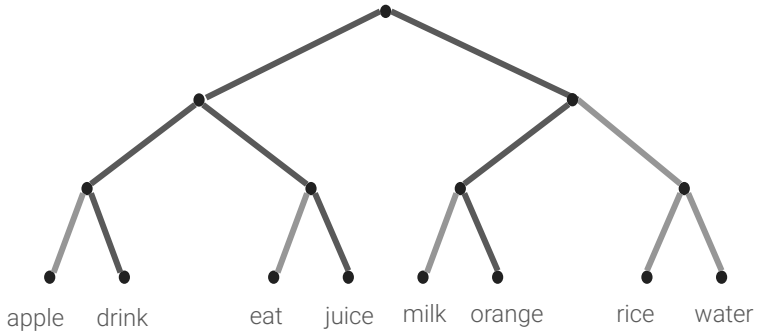
Вычисление softmax на последнем слое требует прохода по всему словарю, т. е. сложность O(n).

Как можно сократить вычислительную сложность?

- Иерархический softmax
- Negative Sampling

Иерархический softmax

- Заменим второй слой сети на какое-нибудь бинарное дерево (например, дерево Хаффмана)
- Каждому листу сопоставим одно из слов словаря
- Каждому внутреннему узлу – свой k-мерный вектор весов



Иерархический softmax

Допустим, мы хотим предсказать слово juice

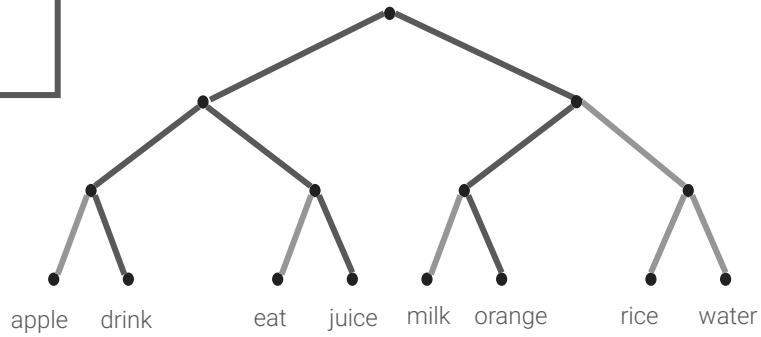
Вероятность перехода их текущей вершины в поддеревья:

$$\boxed{\begin{aligned} &\rho_l = \sigma\left(u_{n(w,0)}^T h\right) \text{ — левое поддерево} \\ &\rho_r = \sigma\left(-u_{n(w,0)}^T h\right) \text{ — правое поддерево} \end{aligned}}$$

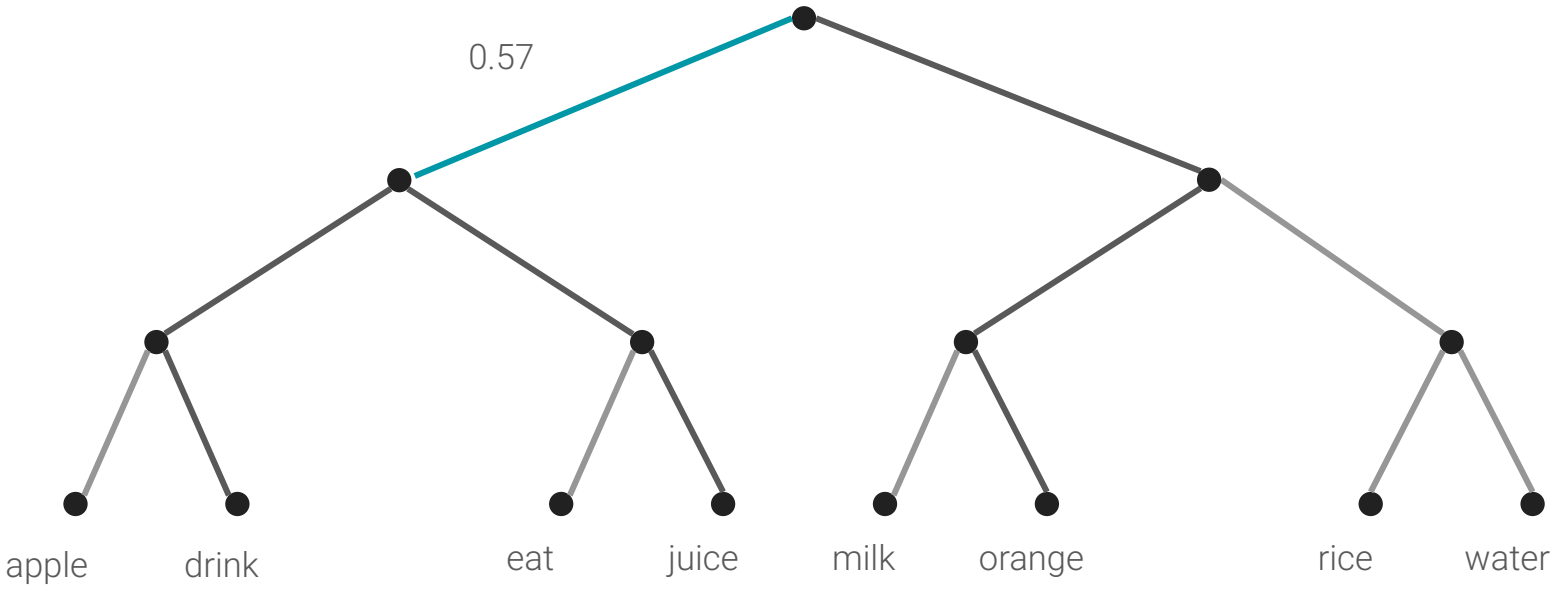
$n(w, i)$ — индекс i — й вершины на пути к слову w

$u_{n(w,\,i)}$ — вектор весов узла i

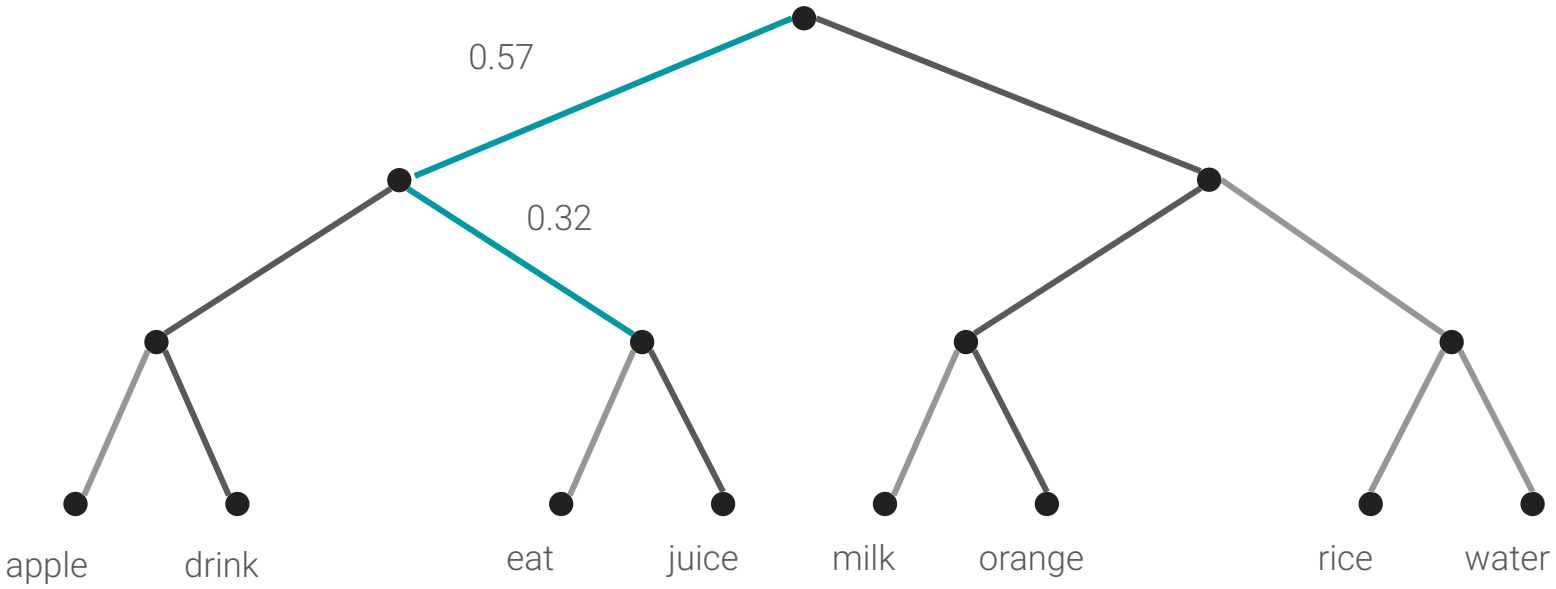
h — скрытое состояние



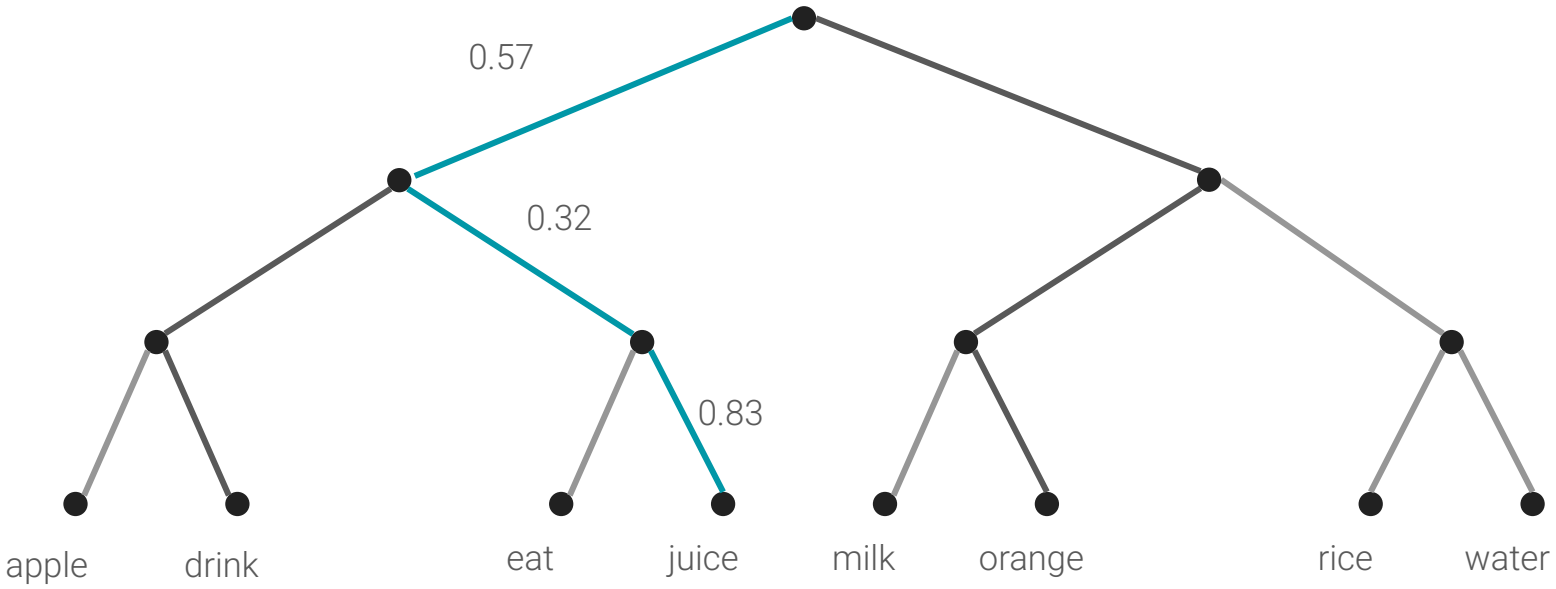
Иерархический softmax



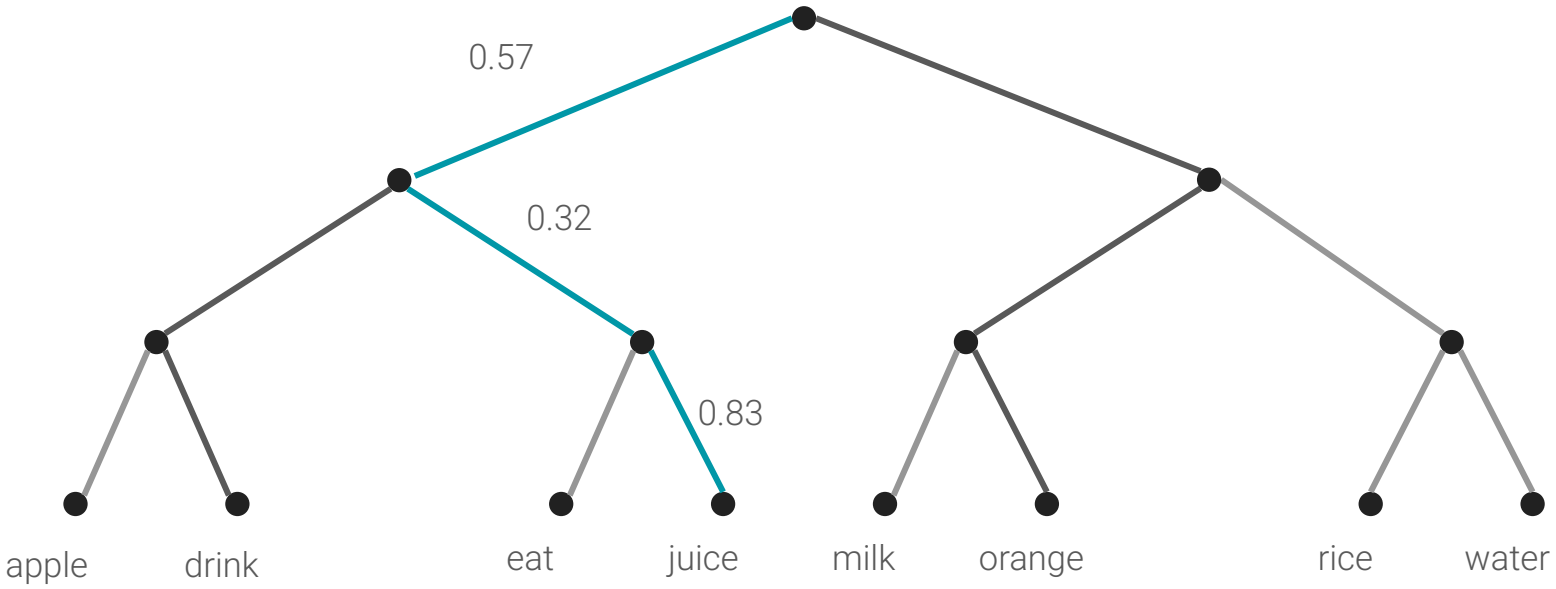
Иерархический softmax



Иерархический softmax

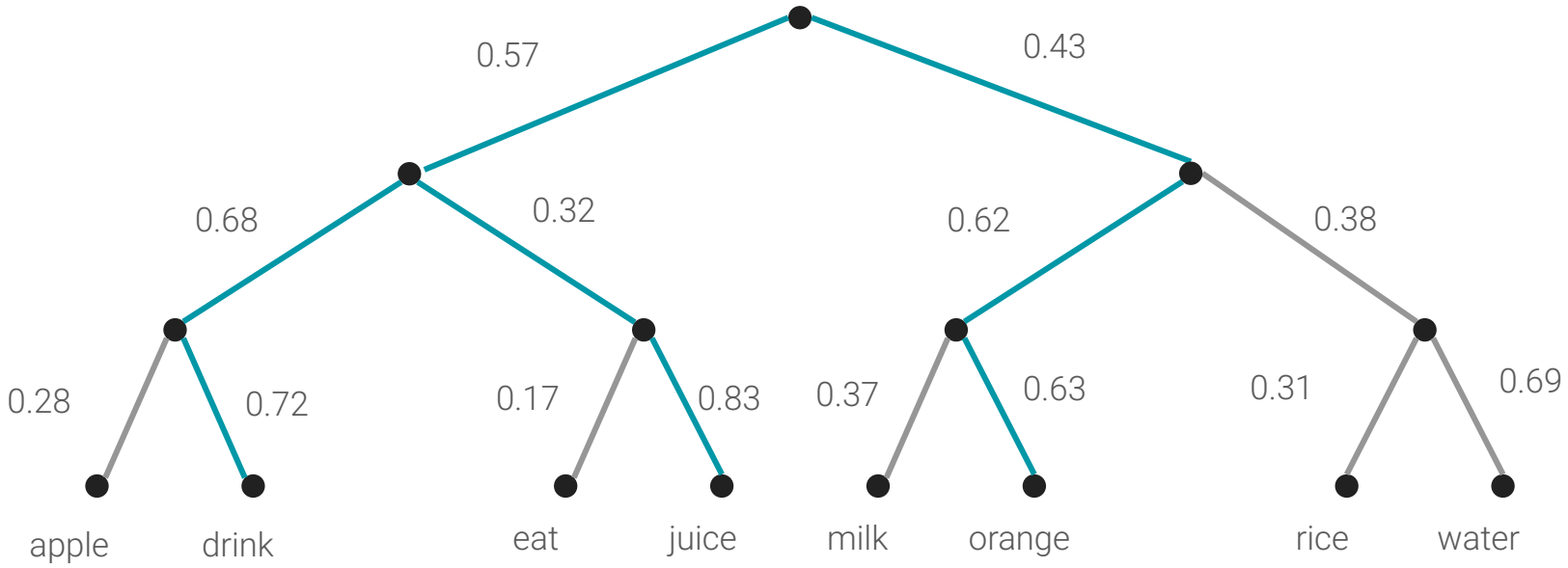


Иерархический softmax

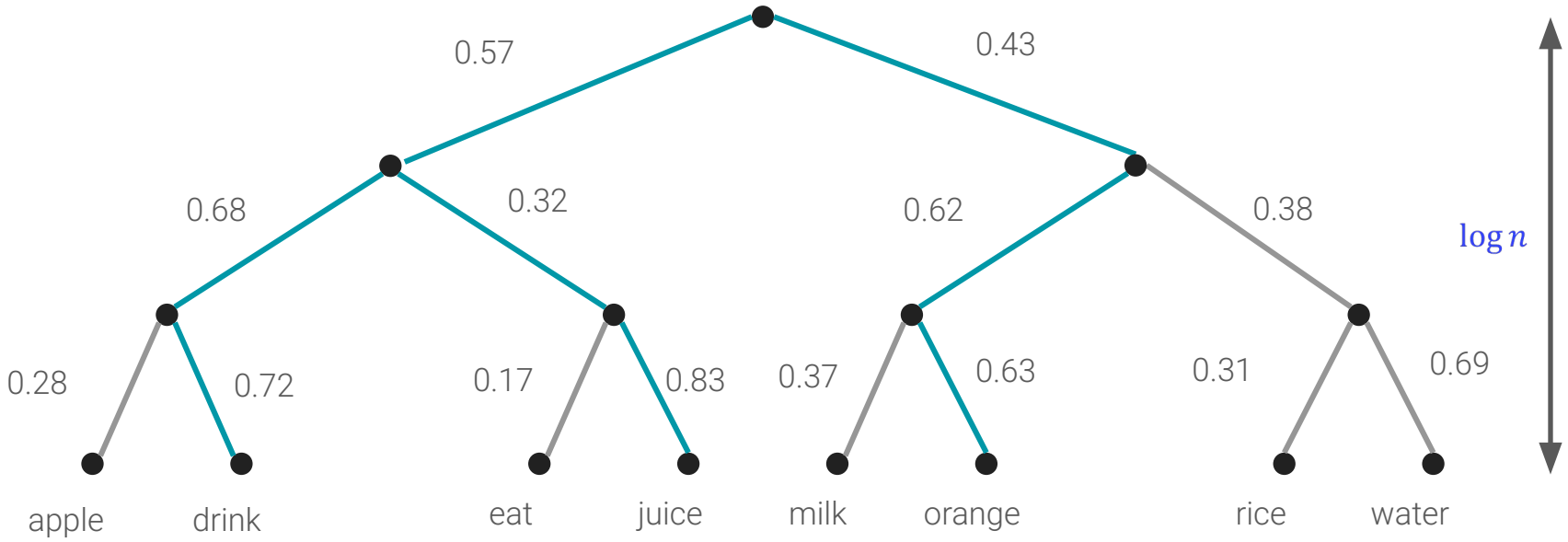


$$\rho(\text{juice}|w) = 0.57 * 0.32 * 0.83 \approx 0.15$$

Иерархический softmax

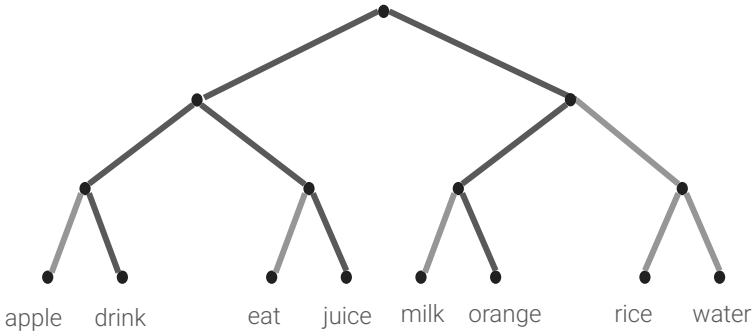


Иерархический softmax



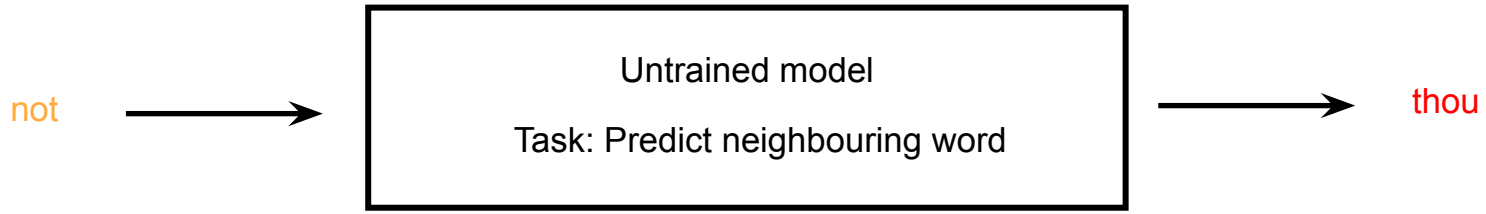
Иерархический softmax

- Получаем h с первого слоя
- Спускаемся по дереву к листьям со словами из контекста
- Получаем вероятность только нужных слов

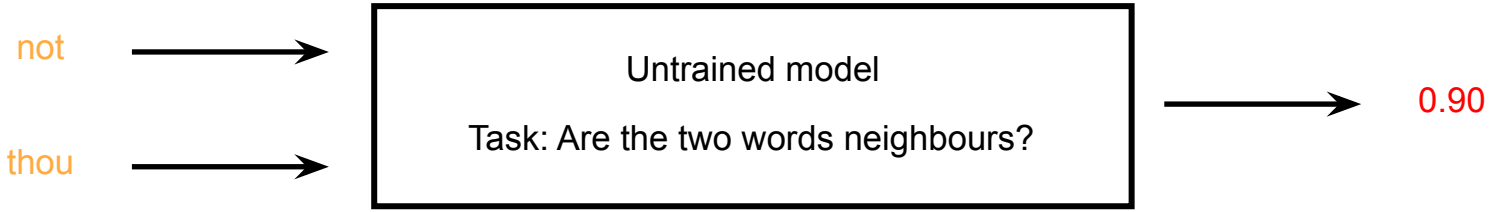


Negative Sampling

Поменяем задачу прогнозирования слова:



На задачу прогнозирования вероятности соседства двух слов:



Negative Sampling

not →

thou →

Smartass model

Task: Are the two words neighbours?

Def model (in, out): return 1.0

input word	output word	target
not	thou	1
not	shalt	1
not	make	1
not	a	1
make	shalt	1
make	not	1
make	a	1
make	machine	1

Negative Sampling

input word	output word	target
not	thou	1
not	aaron	0
not	taco	0
not	shalt	1
not	make	

Pick randomly from vocabulary (random sampling)

Word	Count	Probability
aardvark		
aarhus		
aaron		
taco		
thou		
zyzzyva		

Преимущества Word2vec

- Векторы фиксированной длины.
- Векторы можно один раз обучить и использовать для разных задач.
- Модель учитывает семантику слов, т. е. синонимы будут иметь векторы ближе в пространстве.
- Не требует размеченного текста.

Другие модели векторного представления слов

GloVe (Global Vectors)

- Строим матрицу слово-слово, где компоненты матрицы показывают: сколько раз слово i встретилось в контексте j.

- Вычислим вероятность встречаемости слова i в контексте j, разделив на сумму значений в строке для каждого слова:

Probability and Ratio	$k = solid$	$k = gas$	$k = water$	$k = fashion$
$P(k\|ice)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k\|steam)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k\|ice)/P(k\|steam)$	8.9	8.5×10^{-2}	1.36	0.96

- Обучим векторное представление слов так, чтобы их скалярное произведение равнялось логарифму вероятности совместной встречаемости.

GloVe

Преимущества:

- Простая архитектура без нейронной сети.
- Модель быстрая, и этого может быть достаточно для простых приложений.
- Использует частоту встречаемости слов и опережает Word2Vec на некоторых бенчмарках.
- Осмысленные эмбеддинги.

Недостатки:

- Обучена на уровне слов и даёт немного данных о предложении и контексте, в котором слово используется.
- Плохо обрабатывает неизвестные и редкие слова.

FastText

- Слова разбиваются на последовательности символов — символьные N-граммы.
- Для каждой N-граммы обучается вектор с помощью моделей CBOW или Skip-gram.
- Вектор слова получается усреднением векторов N-грамм.

FastText: методы сокращения N-грамм

- Зафиксируем максимальное число векторов
- Сопоставим им хэш-таблицу
- Несколько N-грамм используют один и тот же вектор

FastText

Преимущества:

- Благодаря N-граммам неплохо работает на редких и устаревших словах, решается проблема OOV.
- Относительно простая архитектура: feed-forward, 1 вход, 1 скрытый слой, один выход (хотя N-граммы добавляют сложность в генерацию эмбеддингов).

Недостатки:

- Для слов с одинаковым написанием, но разными смыслами используется один и тот же вектор.
- Похожие по написанию слова могут путаться.

На этом на сегодня всё